Program 1: Charity Stall Volunteer Scheduling

CSCE 330 — Programming Language Structures

1 Overview

You are given a constraint-satisfaction problem: assign volunteers to shifts at a charity event so that every shift is covered, every volunteer works the right number of shifts, and nobody is scheduled for a shift they aren't available for. Each problem instance is provided as a text file (`puzzle1.pl`, `puzzle2.pl`, ...), and every instance has *exactly one* valid assignment.

Your job is to write a program that reads such a file and finds that assignment. You may use any programming language you like. The solving logic must be your own work, but you are permitted (and encouraged) to use AI to help write the input-parsing portion, subject to the rules in Section 5.

2 The Problem

A charity event has N shifts (about 20) spread across several stalls and time slots, and M volunteers (about 10). Each volunteer has supplied a list of shifts they are available for (between 2 and 5 shifts). You must produce an assignment of one volunteer per shift such that:

(a) Every shift is assigned to exactly one volunteer.

(b) Each volunteer is assigned exactly $K = N/M$ shifts.

(c) A volunteer is only ever assigned to a shift in their availability list.

The instances are constructed so that exactly one assignment satisfies all three constraints. Your program must find it.

3 Input Format

A puzzle file is a plain text file containing Prolog-style facts and comments. Lines starting with `%` are comments and should be ignored. The file contains two kinds of facts:

```
shift(Id, 'Stall Name', 'HH:MM').
available(volunteer_name, [ShiftId1, ShiftId2, ...]).
```

`Id` and the elements of the availability list are integers. `'Stall Name'` and `'HH:MM'` are single-quoted strings. `volunteer_name` is a lowercase identifier (an atom).

A short excerpt from a real puzzle file:

```
% --- Shifts: shift(Id, Stall, Hour) ---
shift(1, 'Cake Stall', '10:00').
shift(2, 'Cake Stall', '11:00').
shift(3, 'Cake Stall', '12:00').
...
```

```
shift(20, 'Crafts', '13:00').

% --- Availabilities: available(Name, [ShiftIds]) ---
available(alice, [1,11,13]).
available(bob, [5,12,16,17,20]).
available(carol, [1,2]).
...
available(jack, [4,16,19]).
```

The reference solution may also be present at the end of the file as a comment block. Your program must **not** read it; you must solve the puzzle from the `shift/3` and `available/2` facts alone.

4 Requirements

1. **Language.** You may use any programming language. Pick something you are comfortable with.
2. **Command-line argument.** Your program must take the path to a puzzle file as its first command-line argument, so I can run it on puzzles you have never seen. For example:

   ```
   $ ./program1 puzzle1.pl
   $ python3 program1.py puzzles/puzzle_grading_07.pl
   ```

3. **Output.** Print the solution as a list of `shift_id` → `volunteer_name` pairs, one per line, in order of shift id. Any clear, human-readable format is fine, for example:

   ```
   Shift 1 (Cake Stall @ 10:00): carol
   Shift 2 (Cake Stall @ 11:00): carol
   ...
   Shift 20 (Crafts @ 13:00): ivy
   ```

4. **Correctness.** On every test instance I run, your output must be the unique valid assignment.
5. **Termination.** Your program must terminate with the correct answer within **3 minutes (wall-clock)** on each test instance. A program that produces correct output but does not finish within the time limit will be treated as non-functional.
6. **No external solvers.** You may not call out to a SAT/CSP library, a Prolog system, an ILP solver, or similar. Write the search yourself. Standard-library data structures (`std::vector`, Python `list`/`dict`, etc.) are fine.

5 Use of AI

You may use an AI assistant (ChatGPT, Claude, Copilot, Gemini, . . .) **only** to help write the *input parsing* portion of your program: the code that opens the file, reads the `shift/3` and `available/2` facts, and stores them in a data structure of your choice. You decide the destination structure — for example, a `std::vector<Shift>` plus a `std::vector<Volunteer>` in C++, or two lists of dictionaries in Python — and tell the AI what shape you want.

The **solving** part (the search / constraint propagation / backtracking / whatever strategy you use) must be your own work.

If you use AI for the parser, you must include in your writeup:

- The exact prompt you gave the AI.
- The exact response you received, verbatim (copy-paste; do not paraphrase).
- A short note on what you changed, if anything, before integrating the code into your program.

If you used AI more than once for the parser (e.g., follow-up prompts to fix a bug), include every prompt and response in order. If you did not use AI at all, state that in your writeup.

6 Deliverables

Submit a single zip or tarball containing:

1. Your source code.
2. A `README` with:
   - How to build your program (if compilation is required).
   - The exact command to run it on a puzzle file.
   - The language and version you used.
3. A `writeup.pdf` (or `.md` / `.txt`) containing:
   - A short description of your solving approach (one or two paragraphs).
   - The AI prompt(s) and response(s) used for parsing, or a statement that no AI was used.
   - Sample output from running on `puzzle1.pl`.

7 Grading

Parser reads the file format correctly	15 %
Program accepts the puzzle file as a CLI argument	5 %
Solver finds the correct assignment on `puzzle1.pl`	25 %
Solver finds the correct assignment on unseen puzzles	35 %
Code quality and `README`	10 %
Writeup (approach + AI prompt/response, if used)	10 %

Notes

- You do not need to handle malformed input. Assume every puzzle file is well-formed and has exactly one solution.
- A brute-force search (try every assignment, check the constraints) will comfortably finish within the 3-minute limit on instances of this size, but a smarter approach (e.g., assign the most-constrained shift first, prune early when a volunteer's quota is met) is much faster and scales better if you want to experiment with larger puzzles.

- Think about what data structure makes your solver easiest to write *before* you ask the AI to build the parser — that decision is the part of the parsing step that's actually yours, and it heavily influences how clean your solver ends up.

Assignment drafted by Claude (claude.ai), with prompting from James O'Reilly.